RAG Pipeline Metrics - Formulas & Calculations

1. Retrieval Metrics

1.1 Precision@K

What: Of the K chunks retrieved, how many are actually relevant?

```
Precision@K = Number of Relevant Chunks in Top-K / K
```

Example:

```
Query: "What is AI?"
Retrieved 5 chunks: [✅ Relevant, ✅ Relevant, ❌ Irrelevant, ✅ Relevant, ❌ Irrelevant]

Precision@5 = 3 / 5 = 0.60
```

Score	Meaning
> 0.8	✅ Excellent
0.6 - 0.8	✅ Good
0.4 - 0.6	⚠️ Needs improvement
< 0.4	❌ Poor retrieval

1.2 Recall@K

What: Of ALL relevant chunks in the database, how many did we retrieve?

```
Recall@K = Number of Relevant Chunks Retrieved / Total Relevant Chunks in Database
```

Example:

```
Database has 10 relevant chunks for "What is AI?"
We retrieved 5 chunks, 3 are relevant

Recall@5 = 3 / 10 = 0.30
```

Score	Meaning
> 0.8	✅ Excellent
0.5 - 0.8	✅ Good
0.3 - 0.5	⚠️ Needs improvement
< 0.3	❌ Missing too many relevant chunks

Trade-off: Higher K → Higher Recall but Lower Precision

1.3 F1 Score

What: Harmonic mean of Precision and Recall — balances both.

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Example:

```
Precision = 0.60
Recall = 0.30

F1 = 2 × (0.60 × 0.30) / (0.60 + 0.30)
F1 = 2 × 0.18 / 0.90
F1 = 0.40
```

Score	Meaning
> 0.7	✅ Good balance
0.5 - 0.7	⚠️ Acceptable
< 0.5	❌ Precision or Recall is too low

1.4 Mean Reciprocal Rank (MRR)

What: How high does the FIRST relevant result appear? Rewards systems that put relevant results at the top.

```
MRR = (1/N) × Σ (1 / rank_of_first_relevant_result)

where N = number of queries
```

Example:

```
Query 1: First relevant chunk at position 1 → 1/1 = 1.00
Query 2: First relevant chunk at position 3 → 1/3 = 0.33
Query 3: First relevant chunk at position 2 → 1/2 = 0.50

MRR = (1/3) × (1.00 + 0.33 + 0.50) = 0.61
```

Score	Meaning
> 0.8	✅ Relevant results consistently at top
0.5 - 0.8	⚠️ Relevant results not always first
< 0.5	❌ Relevant results buried too deep

1.5 Mean Average Precision (MAP)

What: Average of precision values at each relevant result position, averaged across all queries.

```
AP (per query) = (1 / total_relevant) × Σ (Precision@k × is_relevant(k))

MAP = (1/N) × Σ AP_i
```

Example:

```
Query: Retrieved 5 chunks → [✅, ❌, ✅, ❌, ✅]
Total relevant in DB = 3

Position 1: ✅ → Precision@1 = 1/1 = 1.00 ✓
Position 2: ❌ → skip
Position 3: ✅ → Precision@3 = 2/3 = 0.67 ✓
```

```
Position 4: ❌ → skip
Position 5: ✅ → Precision@5 = 3/5 = 0.60 ✓

AP = (1/3) × (1.00 + 0.67 + 0.60) = 0.76

If 3 queries have AP = [0.76, 0.50, 0.90]:
MAP = (0.76 + 0.50 + 0.90) / 3 = 0.72
```

Score	Meaning
> 0.7	✅ Excellent ranking quality
0.5 - 0.7	⚠️ Decent but can improve
< 0.5	❌ Poor ranking

1.6 NDCG (Normalized Discounted Cumulative Gain)

What: Measures ranking quality considering that results at the top matter more than results at the bottom.

```
Relevance scores: rel(i) = relevance of result at position i (e.g., 0, 1, 2, 3)

DCG@K = Σ (rel(i) / log₂(i + 1))    for i = 1 to K

IDCG@K = DCG of the ideal (perfect) ranking

NDCG@K = DCG@K / IDCG@K
```

Example:

```
Retrieved relevance scores: [3, 2, 0, 1, 0]  (3=highly relevant, 0=irrelevant)

DCG@5 = 3/log₂(2) + 2/log₂(3) + 0/log₂(4) + 1/log₂(5) + 0/log₂(6)
      = 3/1.00    + 2/1.58    + 0/2.00    + 1/2.32    + 0/2.58
      = 3.00      + 1.26      + 0.00      + 0.43      + 0.00
      = 4.69

Ideal ranking: [3, 2, 1, 0, 0]
IDCG@5 = 3/1.00 + 2/1.58 + 1/2.00 + 0/2.32 + 0/2.58
       = 3.00   + 1.26   + 0.50   + 0.00   + 0.00
       = 4.76

NDCG@5 = 4.69 / 4.76 = 0.985
```

Score	Meaning
> 0.9	✅ Near-perfect ranking
0.7 - 0.9	✅ Good ranking
0.5 - 0.7	⚠️ Acceptable
< 0.5	❌ Poor ranking quality

1.7 Hit Rate (Hit@K)

What: For what percentage of queries did we find at least ONE relevant chunk?

```
Hit Rate@K = Number of queries with at least 1 relevant result in top-K / Total queries
```

Example:

```
Query 1: Top-5 has relevant chunk → Hit ✅
Query 2: Top-5 has relevant chunk → Hit ✅
Query 3: Top-5 has NO relevant chunk → Miss ❌
Query 4: Top-5 has relevant chunk → Hit ✅

Hit Rate@5 = 3 / 4 = 0.75
```

Score	Meaning
> 0.95	✅ Excellent
0.8 - 0.95	✅ Good
0.6 - 0.8	⚠️ Needs improvement
< 0.6	❌ Retrieval is failing

---

2. Similarity / Distance Metrics

2.1 Cosine Similarity

What: Measures the angle between two vectors. Ignores magnitude, focuses on direction.

```
cosine_sim(A, B) = (A · B) / (||A|| × ||B||)

where:
  A · B = Σ(Aᵢ × Bᵢ)          → dot product
  ||A|| = √(Σ(Aᵢ²))           → magnitude of A
  ||B|| = √(Σ(Bᵢ²))           → magnitude of B
```

Example:

```
A = [1, 2, 3]
B = [2, 4, 6]

A · B = (1×2) + (2×4) + (3×6) = 2 + 8 + 18 = 28
||A|| = √(1 + 4 + 9) = √14 = 3.74
||B|| = √(4 + 16 + 36) = √56 = 7.48

cosine_sim = 28 / (3.74 × 7.48) = 28 / 27.97 = 1.00
```

(These vectors point in the same direction → perfect similarity)

Score	Meaning
0.9 - 1.0	✅ Nearly identical meaning
0.7 - 0.9	✅ Very similar
0.5 - 0.7	⚠️ Somewhat related
0.3 - 0.5	⚠️ Weakly related
< 0.3	❌ Not related

2.2 Euclidean Distance (L2)

What: Straight-line distance between two vectors in space.

```
L2(A, B) = √(Σ(Aᵢ - Bᵢ)²)
```

Example:

```
A = [1, 2, 3]
B = [4, 6, 8]

L2 = √((1-4)² + (2-6)² + (3-8)²)
   = √(9 + 16 + 25)
   = √50
   = 7.07
```

Score	Meaning
0.0 - 0.5	✅ Nearly identical
0.5 - 1.0	✅ Very similar
1.0 - 1.5	⚠️ Moderately similar
1.5 - 2.0	⚠️ Weakly related
> 2.0	❌ Not related

Note: These ranges are approximate and depend on embedding dimensions and normalization.

2.3 Dot Product (Inner Product)

What: Sum of element-wise products. Captures both direction AND magnitude.

```
dot(A, B) = Σ(Aᵢ × Bᵢ)
```

Example:

```
A = [1, 2, 3]
B = [4, 5, 6]

dot = (1×4) + (2×5) + (3×6) = 4 + 10 + 18 = 32
```

Score	Meaning
High positive	✅ Similar direction, large magnitude
Near zero	⚠️ Orthogonal (unrelated)
Negative	❌ Opposite direction

Note: For normalized vectors (||A|| = ||B|| = 1), dot product = cosine similarity.

2.4 Manhattan Distance (L1)

What: Sum of absolute differences. Also called "city block" distance.

```
L1(A, B) = Σ|Aᵢ - Bᵢ|
```

Example:

```
A = [1, 2, 3]
B = [4, 6, 8]

L1 = |1-4| + |2-6| + |3-8| = 3 + 4 + 5 = 12
```

Score	Meaning
Low	✅ Similar
High	❌ Different

---

3. Generation Quality Metrics

3.1 Relevance Score (Keyword-based)

What: How many query keywords appear in the retrieved context?

```
relevance = |query_words ∩ context_words| / |query_words|
```

Example:

```
Query: "What is artificial intelligence"
Query words: {"what", "is", "artificial", "intelligence"}

Context contains: "artificial", "intelligence", "is"

relevance = 3 / 4 = 0.75
```

Score	Meaning
> 0.8	✅ Highly relevant context retrieved
0.5 - 0.8	⚠️ Partially relevant
< 0.5	❌ Context may not answer the question

3.2 Groundedness Score

What: How much of the answer is grounded (supported) by the retrieved context? Detects hallucination.

```
groundedness = |answer_words ∩ context_words| / |answer_words|
```

Example:

```
Answer: "AI is a branch of computer science that simulates human intelligence"
Answer words: {"ai", "is", "a", "branch", "of", "computer", "science", "that", "simulates", "human", "intelligence"}

Context contains: {"ai", "is", "a", "branch", "of", "computer", "science", "human", "intelligence"}
Missing from context: {"that", "simulates"}

groundedness = 9 / 11 = 0.82
```

Score	Meaning
> 0.8	✅ Well grounded, low hallucination risk
0.6 - 0.8	⚠️ Mostly grounded, some new words
0.4 - 0.6	⚠️ Significant content not from context
< 0.4	❌ Likely hallucinating

3.3 Faithfulness (NLI-based)

What: Does the answer logically follow from the context? Uses Natural Language Inference.

```
faithfulness = number_of_claims_supported_by_context / total_claims_in_answer
```

Example:

```
Answer claims:
1. "AI is a branch of computer science" → Context says this ✅
2. "AI was invented in 1956" → Context says this ✅
3. "AI will replace all jobs by 2030" → Context does NOT say this ❌

faithfulness = 2 / 3 = 0.67
```

Score	Meaning
> 0.9	✅ Highly faithful
0.7 - 0.9	⚠️ Mostly faithful
< 0.7	❌ Contains unsupported claims

3.4 BLEU Score

What: Measures n-gram overlap between generated answer and reference answer.

```
BLEU = BP × exp(Σ wₙ × log(pₙ))

where:
  pₙ = modified precision for n-grams
  wₙ = weight (usually 1/N for N-gram BLEU)
  BP = brevity penalty = min(1, exp(1 - ref_length/gen_length))
```

Simplified BLEU-1 (unigram) Example:

```
Reference: "AI is a field of computer science"
Generated: "AI is a branch of computer science"

Matching unigrams: "AI", "is", "a", "of", "computer", "science" = 6
Total generated unigrams: 7

BLEU-1 = 6/7 = 0.857
```

Score	Meaning
> 0.5	✅ Very good (rare for open-ended)
0.3 - 0.5	✅ Good
0.1 - 0.3	⚠️ Acceptable
< 0.1	❌ Poor overlap

Note: BLEU is strict — even good paraphrases score low. Better for translation than RAG.

3.5 ROUGE Scores

What: Measures overlap between generated and reference text. More recall-focused than BLEU.

ROUGE-1 (Unigram)

```
ROUGE-1 Recall = matching_unigrams / total_unigrams_in_reference
ROUGE-1 Precision = matching_unigrams / total_unigrams_in_generated
ROUGE-1 F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

ROUGE-L (Longest Common Subsequence)

```
LCS = length of longest common subsequence

ROUGE-L Recall = LCS / reference_length
ROUGE-L Precision = LCS / generated_length
ROUGE-L F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Example:

```
Reference: "artificial intelligence is a field of computer science"
Generated: "artificial intelligence is a branch of science"

ROUGE-1:
  Matching unigrams: "artificial", "intelligence", "is", "a", "of", "science" = 6
  Recall = 6/8 = 0.75
  Precision = 6/7 = 0.857
  F1 = 2 × (0.75 × 0.857) / (0.75 + 0.857) = 0.80

ROUGE-L:
  LCS: "artificial intelligence is a _ of _ science" → length = 6
  Recall = 6/8 = 0.75
  Precision = 6/7 = 0.857
  F1 = 0.80
```

Score	Meaning
> 0.6	✅ Strong overlap
0.4 - 0.6	⚠️ Moderate overlap
< 0.4	❌ Weak overlap

3.6 BERTScore

What: Uses BERT embeddings to measure semantic similarity (not just word overlap).

```
For each token in generated text, find most similar token in reference (using cosine similarity):

Precision = (1/|generated|) × Σ max_cosine_sim(gen_token, ref_tokens)
Recall = (1/|reference|) × Σ max_cosine_sim(ref_token, gen_tokens)
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Why better than BLEU/ROUGE?

```
Reference: "The car is fast"
Generated: "The automobile is speedy"

BLEU/ROUGE: Low score (different words)
BERTScore: High score (same meaning — "car"≈"automobile", "fast"≈"speedy")
```

Score	Meaning
> 0.9	✅ Semantically very similar
0.8 - 0.9	✅ Good semantic match

Score	Meaning
0.7 - 0.8	⚠️ Moderate
< 0.7	❌ Semantically different

---

4. End-to-End / System Metrics

4.1 Answer Correctness

What: Is the final answer factually correct?

```
answer_correctness = correct_answers / total_questions
```

Score	Meaning
> 0.9	✅ Production ready
0.8 - 0.9	⚠️ Good but needs tuning
< 0.8	❌ Not reliable enough

4.2 Latency

What: Time from user query to final answer.

```
total_latency = embedding_time + search_time + llm_time + post_processing_time
```

Typical Breakdown:

```
Embedding (query):     10-200ms
Vector Search:         1-50ms
Re-ranking:            10-100ms
LLM Generation:        500-5000ms
Post-processing:       1-10ms
──────────────────────────────
Total:                 ~600-5000ms
```

Latency	Meaning
< 1s	✅ Excellent (real-time feel)
1-3s	✅ Good (acceptable for most apps)
3-5s	⚠️ Noticeable delay
5-10s	⚠️ Needs optimization
> 10s	❌ Too slow for interactive use

4.3 Token Usage & Cost

```
cost_per_query = (input_tokens × input_price) + (output_tokens × output_price)
```

Example (GPT-4o-mini):

```
Input: 1500 tokens (prompt + context) × $0.00000015 = $0.000225
Output: 200 tokens (answer) × $0.0000006 = $0.000120
```

```
──────────────────────────────────────────────────
Total per query: $0.000345 ≈ $0.0003

1000 queries/day = $0.30/day = $9/month
```

Cost/Query	Meaning
< $0.001	✅ Very cheap
$0.001 - $0.01	✅ Reasonable
$0.01 - $0.10	⚠️ Getting expensive
> $0.10	❌ Optimize or switch model

4.4 Chunk Coverage

What: How many retrieved chunks actually contributed to the answer?

```
coverage = chunks_contributing_to_answer / total_chunks_retrieved
```

Example:

```
Retrieved 5 chunks
Chunk 1: Keywords found in answer ✅
Chunk 2: Keywords found in answer ✅
Chunk 3: No keywords in answer ❌
Chunk 4: Keywords found in answer ✅
Chunk 5: No keywords in answer ❌

coverage = 3 / 5 = 0.60
```

Score	Meaning
> 0.7	✅ Most chunks are useful
0.4 - 0.7	⚠️ Some chunks wasted
< 0.4	❌ Retrieving too many irrelevant chunks, reduce K

---

5. RAGAS Framework Metrics

5.1 Context Precision

What: Are the relevant chunks ranked higher than irrelevant ones?

```
Context Precision@K = (1/K) × Σ (Precision@k × is_relevant(k))
```

Score	Meaning
> 0.8	✅ Relevant chunks at top
< 0.5	❌ Relevant chunks buried

5.2 Context Recall

What: Can all claims in the ground truth answer be attributed to retrieved context?

```
Context Recall = claims_attributable_to_context / total_claims_in_ground_truth
```

Score	Meaning
> 0.8	✅ Context covers the answer well
< 0.5	❌ Missing important context

5.3 Answer Similarity

What: Semantic similarity between generated answer and ground truth.

```
Answer Similarity = cosine_similarity(embed(generated_answer), embed(ground_truth))
```

Score	Meaning
> 0.8	✅ Very close to expected answer
0.6 - 0.8	⚠️ Partially correct
< 0.6	❌ Significantly different

---

6. Quick Reference: All Metrics Summary

Metric	Category	Range	Direction	Good Score	Formula
Precision@K	Retrieval	0-1	Higher ✅	> 0.7	relevant_in_K / K
Recall@K	Retrieval	0-1	Higher ✅	> 0.8	relevant_in_K / total_relevant
F1	Retrieval	0-1	Higher ✅	> 0.7	2×P×R / (P+R)
MRR	Retrieval	0-1	Higher ✅	> 0.7	mean(1/rank_first_relevant)
MAP	Retrieval	0-1	Higher ✅	> 0.6	mean(average_precision)
NDCG	Retrieval	0-1	Higher ✅	> 0.7	DCG / IDCG
Hit Rate	Retrieval	0-1	Higher ✅	> 0.9	queries_with_hit / total
Cosine Sim	Similarity	-1 to 1	Higher ✅	> 0.7	A·B / (‖A‖×‖B‖)
L2 Distance	Similarity	0 to ∞	Lower ✅	< 1.0	$\sqrt{\Sigma(A-B)^2}$
Dot Product	Similarity	-∞ to ∞	Higher ✅	Depends	Σ(A×B)
Relevance	Generation	0-1	Higher ✅	> 0.8	query∩context / query
Groundedness	Generation	0-1	Higher ✅	> 0.7	answer∩context / answer
Faithfulness	Generation	0-1	Higher ✅	> 0.85	supported_claims / total_claims
BLEU	Generation	0-1	Higher ✅	> 0.3	n-gram precision with BP
ROUGE-L	Generation	0-1	Higher ✅	> 0.4	LCS-based F1
BERTScore	Generation	0-1	Higher ✅	> 0.8	BERT embedding similarity
Latency	System	0 to ∞	Lower ✅	< 3s	total pipeline time
Cost/Query	System	0 to ∞	Lower ✅	< $0.01	tokens × price
Coverage	System	0-1	Higher ✅	> 0.5	used_chunks / retrieved_chunks

---